



university of
 groningen

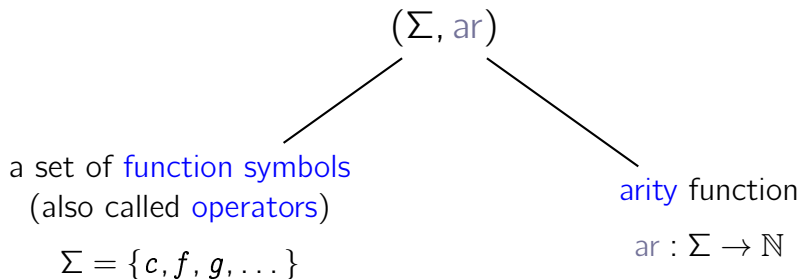
Models and Semantics of Computation (MaSC)

Lecture 4: IMP

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

Lecture 2: Unsorted Signature



Each symbol has an arity. Examples:

- ▶ $\text{ar}(c) = 0$: constant
- ▶ $\text{ar}(f) = 1$: unary
- ▶ $\text{ar}(g) = 2$: binary
- ▶ $\text{ar}(h) = 3$: ternary

Terms Over a Signature



Consider given:

$$\Sigma = \{\Sigma_n\}_{n \in \mathbb{N}}$$

a signature

$$X = \{x, y, z, \dots\}$$

an infinite set of **variables**

Let $T_{\Sigma, X}$ denote the set of all **terms** over Σ, X . The least set such that:

- ▶ if $x \in X$, then $x \in T_{\Sigma, X}$
- ▶ if $c \in \Sigma_0$, then $c \in T_{\Sigma, X}$
- ▶ if $f \in \Sigma_n$ and $t_1, \dots, t_n \in T_{\Sigma, X}$, then $f(t_1, \dots, t_n) \in T_{\Sigma, X}$

Terms Over a Signature



Consider given:

$$\Sigma = \{\Sigma_n\}_{n \in \mathbb{N}}$$

a signature

$$X = \{x, y, z, \dots\}$$

an infinite set of **variables**

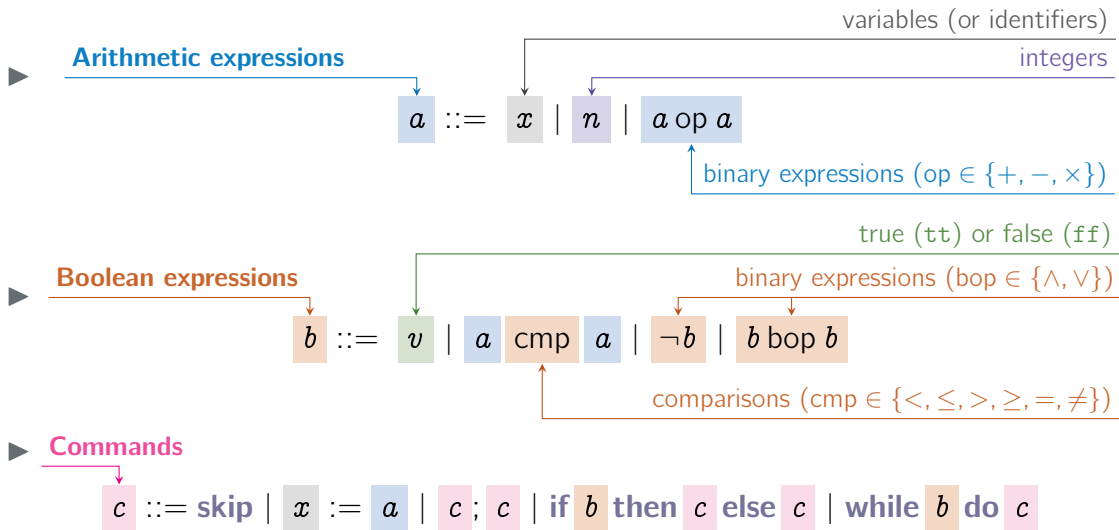
Let $T_{\Sigma, X}$ denote the set of all **terms** over Σ, X . The least set such that:

- ▶ if $x \in X$, then $x \in T_{\Sigma, X}$
- ▶ if $c \in \Sigma_0$, then $c \in T_{\Sigma, X}$
- ▶ if $f \in \Sigma_n$ and $t_1, \dots, t_n \in T_{\Sigma, X}$, then $f(t_1, \dots, t_n) \in T_{\Sigma, X}$

Put differently:

$$T_{\Sigma, X} \ni t ::= \underbrace{x}_{x \in X} \mid \underbrace{c}_{c \in \Sigma_0} \mid \underbrace{f(t_1, \dots, t_n)}_{f \in \Sigma_n}$$

Lecture 3: The Language IMP



Requirements



- ▶ In the previous slide, what do colors actually mean?

Requirements



- ▶ In the previous slide, what do colors actually mean?
- ▶ Clearly, there are strings that we don't want:
 - ▶ $\neg 1985$ (negation only defined on booleans)
 - ▶ $\text{apple} \leq \text{orange}$ (comparisons only defined on arithmetic exps)
 - ▶ $41; 42$ (concatenation only defined on commands)
 - ▶ **while** apple **do** $x := x + tt$ (loops only defined on boolean conditions)

Requirements



- ▶ In the previous slide, what do colors actually mean?
- ▶ Clearly, there are strings that we don't want:
 - ▶ $\neg 1985$ (negation only defined on booleans)
 - ▶ $\text{apple} \leq \text{orange}$ (comparisons only defined on arithmetic exprs)
 - ▶ $41; 42$ (concatenation only defined on commands)
 - ▶ **while** apple **do** $x := x + tt$ (loops only defined on boolean conditions)
- ▶ Beyond excluding nonsensical terms, we want to give formal semantics exclusively to terms that make sense

Requirements



- ▶ In the previous slide, what do colors actually mean?
- ▶ Clearly, there are strings that we don't want:
 - ▶ $\neg 1985$ (negation only defined on booleans)
 - ▶ $\text{apple} \leq \text{orange}$ (comparisons only defined on arithmetic exprs)
 - ▶ $41; 42$ (concatenation only defined on commands)
 - ▶ **while** apple **do** $x := x + tt$ (loops only defined on boolean conditions)
- ▶ Beyond excluding nonsensical terms, we want to give formal semantics exclusively to terms that make sense
- ▶ “terms that make sense” = **well-sorted terms**

Terms Over a Signature



Assume:

$S = \{s_1, s_2, \dots\}$ a set of **sorts** (think: types)
 $\Sigma = \{\Sigma_{s_1, \dots, s_n, s}\}_{s_1, \dots, s_n, s \in S}$ a **many-sorted signature**

Terms Over a Signature



Assume:

$S = \{s_1, s_2, \dots\}$ a set of **sorts** (think: types)
 $\Sigma = \{\Sigma_{s_1, \dots, s_n, s}\}_{s_1, \dots, s_n, s \in S}$ a **many-sorted signature**

► This way, we have

$$f \in \Sigma_{s_1, \dots, s_n, s} \quad f : (s_1 \times \dots \times s_n) \rightarrow s$$

Terms Over a Signature



Assume:

$$\begin{aligned} S &= \{s_1, s_2, \dots\} && \text{a set of sorts (think: types)} \\ \Sigma &= \{\Sigma_{s_1, \dots, s_n, s}\}_{s_1, \dots, s_n, s \in S} && \text{a many-sorted signature} \end{aligned}$$

► This way, we have

$$f \in \Sigma_{s_1, \dots, s_n, s} \quad f : (s_1 \times \dots \times s_n) \rightarrow s$$

- The set of all terms of sort s , denoted $T_{\Sigma, s}$, is the least set such that:
- (constants) if $c \in \Sigma_{\epsilon, s}$ then $c \in T_{\Sigma, s}$
 - (operators) if $f \in \Sigma_{s_1, \dots, s_n, s}$ and $\forall i. t_i \in T_{\Sigma, s_i}$ then $f(t_1, \dots, t_n) \in T_{\Sigma, s}$

Terms Over a Signature



Assume:

$$\begin{aligned} S &= \{s_1, s_2, \dots\} && \text{a set of sorts (think: types)} \\ \Sigma &= \{\Sigma_{s_1, \dots, s_n, s}\}_{s_1, \dots, s_n, s \in S} && \text{a many-sorted signature} \end{aligned}$$

- This way, we have

$$f \in \Sigma_{s_1, \dots, s_n, s} \quad f : (s_1 \times \dots \times s_n) \rightarrow s$$

- The set of all terms of sort s , denoted $T_{\Sigma, s}$, is the least set such that:
 - (constants) if $c \in \Sigma_{\epsilon, s}$ then $c \in T_{\Sigma, s}$
 - (operators) if $f \in \Sigma_{s_1, \dots, s_n, s}$ and $\forall i. t_i \in T_{\Sigma, s_i}$ then $f(t_1, \dots, t_n) \in T_{\Sigma, s}$
- $T_{\Sigma} = \{T_{\Sigma, s}\}_{s \in S}$ denotes the set of all well-sorted terms

Today



- ▶ IMP with sorts (i.e. well-sorted imperative terms)
- ▶ Semantics of IMP (big-step)
- ▶ A termination property (expressions)
- ▶ Memories and commands
- ▶ A termination property (commands)
- ▶ Divergence
- ▶ Structural induction for commands

Last Time: Arithmetic Expressions



$$x \in \text{Ide} \quad n \in \mathbb{Z} \quad \text{op} \in \{+, -, \times\}$$

$$a \in \text{Aexp} ::= x \mid n \mid a \text{ op } a$$

Last Time: Arithmetic Expressions



$$x \in \text{Ide} \quad n \in \mathbb{Z} \quad \text{op} \in \{+, -, \times\}$$

$$a \in \text{Aexp} ::= x \mid n \mid a \text{ op } a$$

$$\mathcal{S} \triangleq \{\text{Aexp}\}$$

$$\Sigma_{\epsilon, \text{Aexp}} \triangleq \text{Ide} \cup \mathbb{Z}$$

$$\Sigma_{\text{Aexp} \text{ Aexp}, \text{Aexp}} \triangleq \{+, \times, -\}$$

Boolean Expressions



$$\begin{array}{lll} x \in \text{Ide} & n \in \mathbb{Z} & v \in \mathbb{B} \\ \text{op} \in \{+, -, \times\} & \text{bop} \in \{\wedge, \vee\} & \text{cmp} \in \{<, \leq, >, \geq, =, \neq\} \end{array}$$

$$\begin{array}{l} a \in Aexp ::= x \mid n \mid a \text{ op } a \\ b \in Bexp ::= v \mid a \text{ cmp } a \mid \neg b \mid b \text{ bop } b \end{array}$$

$$S \triangleq \{Aexp, Bexp\}$$

$$\begin{array}{ll} \Sigma_{\epsilon, Aexp} \triangleq \text{Ide} \cup \mathbb{Z} & \Sigma_{Aexp \ Aexp, Aexp} \triangleq \{+, \times, -\} \\ \Sigma_{\epsilon, Bexp} \triangleq \mathbb{B} & \Sigma_{Aexp \ Aexp, Bexp} \triangleq \{<, \leq, >, \geq, =, \neq\} \\ \Sigma_{Bexp, Bexp} \triangleq \{\neg\} & \Sigma_{Bexp \ Bexp, Bexp} \triangleq \{\vee, \wedge\} \end{array}$$

IMP Semantics (Expressions)



$$\frac{}{\langle x, \sigma \rangle \longrightarrow \sigma(x)} \quad \frac{}{\langle n, \sigma \rangle \longrightarrow n} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow n_0 \text{ op } n_1}$$

$$\frac{}{\langle v, \sigma \rangle \longrightarrow v} \quad \frac{\langle b, \sigma \rangle \longrightarrow v}{\langle \neg b, \sigma \rangle \longrightarrow \neg v} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$

$$\frac{\langle b_0, \sigma \rangle \longrightarrow v_0 \quad \langle b_1, \sigma \rangle \longrightarrow v_1}{\langle b_0 \text{ bop } b_1, \sigma \rangle \longrightarrow v_0 \text{ bop } v_1}$$

IMP Semantics (Expressions)



$$\frac{}{\langle x, \sigma \rangle \longrightarrow \sigma(x)} \quad \frac{}{\langle n, \sigma \rangle \longrightarrow n} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow n_0 \text{ op } n_1}$$

$$\frac{}{\langle v, \sigma \rangle \longrightarrow v} \quad \frac{\langle b, \sigma \rangle \longrightarrow v}{\langle \neg b, \sigma \rangle \longrightarrow \neg v} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$
$$\frac{\langle b_0, \sigma \rangle \longrightarrow v_0 \quad \langle b_1, \sigma \rangle \longrightarrow v_1}{\langle b_0 \text{ bop } b_1, \sigma \rangle \longrightarrow v_0 \text{ bop } v_1}$$

These rules define two different **well-formed formulas**:

- ▶ $\langle a, \sigma \rangle \longrightarrow n$, with $a \in Aexp$, $n \in \mathbb{Z}$
- ▶ $\langle b, \sigma \rangle \longrightarrow v$, with $b \in Bexp$, $v \in \mathbb{B}$

IMP Semantics (Expressions)



$$\frac{}{\langle x, \sigma \rangle \longrightarrow \sigma(x)} \quad \frac{}{\langle n, \sigma \rangle \longrightarrow n} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow n_0 \text{ op } n_1}$$

$$\frac{}{\langle v, \sigma \rangle \longrightarrow v} \quad \frac{\langle b, \sigma \rangle \longrightarrow v}{\langle \neg b, \sigma \rangle \longrightarrow \neg v} \quad \frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$
$$\frac{\langle b_0, \sigma \rangle \longrightarrow v_0 \quad \langle b_1, \sigma \rangle \longrightarrow v_1}{\langle b_0 \text{ bop } b_1, \sigma \rangle \longrightarrow v_0 \text{ bop } v_1}$$

A **termination property** (expanded):

- ▶ $P(b) \triangleq \forall \sigma \in \mathbb{M}. \exists v \in \mathbb{B}. \langle b, \sigma \rangle \longrightarrow v$
- ▶ $\forall b. P(b)?$

Termination: Base Case



$$\forall v \in \mathbb{B}. P(v)$$



Termination: Base Case



$$\forall v \in \mathbb{B}. P(v)$$

Take some arbitrary $v \in \mathbb{B}$. We must prove:

$$P(v) \triangleq \forall \sigma. \exists u. \langle v, \sigma \rangle \longrightarrow u$$



Termination: Base Case



$$\forall v \in \mathbb{B}. P(v)$$

Take some arbitrary $v \in \mathbb{B}$. We must prove:

$$P(v) \triangleq \forall \sigma. \exists u. \langle v, \sigma \rangle \longrightarrow u$$

- Let $\sigma \in \mathbb{M}$ be some arbitrary memory.
Consider the goal $\langle v, \sigma \rangle \longrightarrow u$, where u is the only variable.

Termination: Base Case



$$\forall v \in \mathbb{B}. P(v)$$

Take some arbitrary $v \in \mathbb{B}$. We must prove:

$$P(v) \triangleq \forall \sigma. \exists u. \langle v, \sigma \rangle \longrightarrow u$$

- ▶ Let $\sigma \in \mathbb{M}$ be some arbitrary memory.
Consider the goal $\langle v, \sigma \rangle \longrightarrow u$, where u is the only variable.
- ▶ By rule $\frac{}{\langle v, \sigma \rangle \longrightarrow v}$ we have $\langle v, \sigma \rangle \longrightarrow u \nwarrow_{[u=v]} \square$

Termination: Base Case



$$\forall v \in \mathbb{B}. P(v)$$

Take some arbitrary $v \in \mathbb{B}$. We must prove:

$$P(v) \triangleq \forall \sigma. \exists u. \langle v, \sigma \rangle \longrightarrow u$$

- ▶ Let $\sigma \in \mathbb{M}$ be some arbitrary memory.
Consider the goal $\langle v, \sigma \rangle \longrightarrow u$, where u is the only variable.
- ▶ By rule $\frac{}{\langle v, \sigma \rangle \longrightarrow v}$ we have $\langle v, \sigma \rangle \longrightarrow u \nwarrow_{[u=v]} \square$
- ▶ We are done by taking $u = v$.

Termination: Another Case



$$\forall a_0, a_1. P(a_0 \text{ cmp } a_1)$$



Termination: Another Case



$$\forall a_0, a_1. P(a_0 \text{ cmp } a_1)$$

Take some arbitrary a_0, a_1 . We must prove:

$$P(a_0 \text{ cmp } a_1) \triangleq \forall \sigma. \exists v. \langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v$$

► Let $\sigma \in \mathbb{M}$ be some memory. Consider the goal $\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow v$.

Termination: Another Case



$$\forall a_0, a_1. P(a_0 \text{ cmp } a_1)$$

Take some arbitrary a_0, a_1 . We must prove:

$$P(a_0 \text{ cmp } a_1) \triangleq \forall \sigma. \exists v. \langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v$$

- ▶ Let $\sigma \in \mathbb{M}$ be some memory. Consider the goal $\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow v$.
- ▶ By rule
$$\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$
 we have
$$\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v \nwarrow_{[v=n_0 \text{ cmp } n_1]} \langle a_0, \sigma \rangle \longrightarrow n_0, \langle a_1, \sigma \rangle \longrightarrow n_1$$

Termination: Another Base Case



$$\forall a_0, a_1. P(a_0 \text{ cmp } a_1)$$

Take some arbitrary a_0, a_1 . We must prove:

$$P(a_0 \text{ cmp } a_1) \triangleq \forall \sigma. \exists v. \langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v$$

- ▶ Let $\sigma \in \mathbb{M}$ be some memory. Consider the goal $\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow v$.
- ▶ By rule
$$\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$
 we have
$$\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v \nwarrow_{[v=n_0 \text{ cmp } n_1]} \langle a_0, \sigma \rangle \longrightarrow n_0, \langle a_1, \sigma \rangle \longrightarrow n_1$$
- ▶ By the **termination of arithmetic expressions** (not IH!), such n_0, n_1 exist

Termination: Another Base Case



$$\forall a_0, a_1. P(a_0 \text{ cmp } a_1)$$

Take some arbitrary a_0, a_1 . We must prove:

$$P(a_0 \text{ cmp } a_1) \triangleq \forall \sigma. \exists v. \langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v$$

- ▶ Let $\sigma \in \mathbb{M}$ be some memory. Consider the goal $\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow v$.
- ▶ By rule
$$\frac{\langle a_0, \sigma \rangle \longrightarrow n_0 \quad \langle a_1, \sigma \rangle \longrightarrow n_1}{\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow n_0 \text{ cmp } n_1}$$
 we have
$$\langle a_0 \text{ cmp } a_1, \sigma \rangle \longrightarrow v \nwarrow_{[v=n_0 \text{ cmp } n_1]} \langle a_0, \sigma \rangle \longrightarrow n_0, \langle a_1, \sigma \rangle \longrightarrow n_1$$
- ▶ By the **termination of arithmetic expressions** (not IH!), such n_0, n_1 exist
- ▶ We are done by taking $v = n_0 \text{ cmp } n_1$.

Termination: Inductive Cases



$$\forall b. P(b) \Rightarrow P(\neg b)$$

$$\forall b_0, b_1. (P(b_0) \wedge P(b_1)) \Rightarrow P(b_0 \text{ bop } b_1)$$

Exercise: Complete these cases yourself.

Commands



$a \in Aexp ::= x \mid n \mid a \text{ op } a \quad b \in Bexp ::= v \mid a \text{ cmp } a \mid \neg b \mid b \text{ bop } b$
 $c \in Com ::= \text{skip} \mid x := a \mid c; c \mid \text{if } b \text{ then } c \text{ else } c \mid \text{while } b \text{ do } c$

$$S \triangleq \{Aexp, Bexp, Com\}$$

$$\Sigma_{\epsilon, Aexp} \triangleq \text{Ide} \cup \mathbb{Z}$$

$$\Sigma_{Aexp \ Aexp, Aexp} \triangleq \{+, \times, -\}$$

$$\Sigma_{\epsilon, Bexp} \triangleq \mathbb{B}$$

$$\Sigma_{Aexp \ Aexp, Bexp} \triangleq \{<, \leq, >, \geq, =, \neq\}$$

$$\Sigma_{Bexp, Bexp} \triangleq \{\neg\}$$

$$\Sigma_{Bexp \ Bexp, Bexp} \triangleq \{\vee, \wedge\}$$

$$\Sigma_{\epsilon, Com} \triangleq \{\text{skip}\}$$

$$\Sigma_{Aexp, Com} \triangleq \{x ::= \mid x \in \text{Ide}\}$$

$$\Sigma_{Com \ Com, Com} \triangleq \{;\}$$

$$\Sigma_{Bexp \ Com \ Com, Com} \triangleq \{\text{if} \cdot \text{then} \cdot \text{else} \cdot\}$$

$$\Sigma_{Bexp \ Com, Com} \triangleq \{\text{while} \cdot \text{do} \cdot\}$$

Memories



- A mapping from identifiers (locations / variables) to integers:

$$\mathbb{M} \triangleq \{ \sigma : \text{Ide} \rightarrow \mathbb{Z} \mid \sigma(x) \neq 0 \}$$

σ has finite support

Memories



- A mapping from identifiers (locations / variables) to integers:

$$\mathbb{M} \triangleq \{ \sigma : \text{Ide} \rightarrow \mathbb{Z} \mid \sigma(x) \neq 0 \}$$

↑ σ has finite support

- Example: Memory $\sigma = (42/x, 37/y)$ is defined only for x and y .
We write $\sigma(x) = 42$ and $\sigma(y) = 37$. For any other $z \in \text{Ide}$, $\sigma(z) = 0$.

Memories



- A mapping from identifiers (locations / variables) to integers:

$$\mathbb{M} \triangleq \{ \sigma : \text{Ide} \rightarrow \mathbb{Z} \mid \sigma(x) \neq 0 \}$$

↑ σ has finite support

- Example: Memory $\sigma = (42/x, 37/y)$ is defined only for x and y .
We write $\sigma(x) = 42$ and $\sigma(y) = 37$. For any other $z \in \text{Ide}$, $\sigma(z) = 0$.
- Formally:

$$(n_1/x_1, \dots, n_k/x_k) : \text{Ide} \rightarrow \mathbb{Z}$$

↑ all different ↑

$$(n_1/x_1, \dots, n_k/x_k)(x) \triangleq \begin{cases} n_i & \text{if } x = x_i, \\ 0 & \text{otherwise.} \end{cases}$$

Memories



- A mapping from identifiers (locations / variables) to integers:

$$\mathbb{M} \triangleq \{ \sigma : \text{Ide} \rightarrow \mathbb{Z} \mid \sigma(x) \neq 0 \}$$

↑ σ has finite support

- Example: Memory $\sigma = (42/x, 37/y)$ is defined only for x and y . We write $\sigma(x) = 42$ and $\sigma(y) = 37$. For any other $z \in \text{Ide}$, $\sigma(z) = 0$.
- Formally:

$$(n_1/x_1, \dots, n_k/x_k) : \text{Ide} \rightarrow \mathbb{Z}$$

↑ all different ↑

$$(n_1/x_1, \dots, n_k/x_k)(x) \triangleq \begin{cases} n_i & \text{if } x = x_i, \\ 0 & \text{otherwise.} \end{cases}$$

- The initial memory is $\sigma_0 \triangleq ()$.



$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$



$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$

Example

Suppose $\sigma = (42/x, 37/y)$.

- $\sigma[74/y]$ represents an update of σ : a revised mapping.
Then $\sigma[74/y](y) = 74$ (actual update) and $\sigma[74/y](x) = 42$ (as before)



$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$



$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$

Some properties:

1. $\forall \sigma, m, n, y. \sigma[m/y][n/y] = \sigma[n/y]$. We have:

$$\sigma[m/y][n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma[m/y](x) = \sigma(x) & \text{otherwise.} \end{cases}$$

$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$

Some properties:

1. $\forall \sigma, m, n, y. \sigma[m/y][n/y] = \sigma[n/y]$. We have:

$$\sigma[m/y][n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma[m/y](x) = \sigma(x) & \text{otherwise.} \end{cases}$$

2. $\forall \sigma, m, n, y, z. y \neq z \Rightarrow \sigma[n/y][m/z] = \sigma[m/z][n/y]$.

We use the simpler notation $\sigma[n/y, m/z]$ in such cases.

$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma(x) & \text{otherwise.} \end{cases}$$

Some properties:

1. $\forall \sigma, m, n, y. \sigma[m/y][n/y] = \sigma[n/y]$. We have:

$$\sigma[m/y][n/y](x) \triangleq \begin{cases} n & \text{if } x = y, \\ \sigma[m/y](x) = \sigma(x) & \text{otherwise.} \end{cases}$$

2. $\forall \sigma, m, n, y, z. y \neq z \Rightarrow \sigma[n/y][m/z] = \sigma[m/z][n/y]$.
We use the simpler notation $\sigma[n/y, m/z]$ in such cases.
3. Also: $(n_1/x_1, \dots, n_k/x_k) = \sigma_0[n_1/x_1, \dots, n_k/x_k]$.

IMP Semantics (Commands)



$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \longrightarrow \sigma} \quad \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]} \quad \frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'} \\[2ex] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff} \quad \langle c_1, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \quad \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c_0, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \\[2ex] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma} \\[2ex] \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma'} \end{array}$$

Here the **well-formed formulas** defined by the rules are of the form $\langle c, \sigma \rangle \longrightarrow \sigma'$.

Example



A command c :

$$\underbrace{x := 0; \text{while } 0 \leq y \text{ do } \underbrace{x := x + (2 \times y) + 1; y := y - 1}_{c_3}}_{c_2}}_{c_1}}_c$$

What does c do?

Example



A command c :

$$\underbrace{\underbrace{x := 0; \text{while } 0 \leq y \text{ do } \underbrace{x := x + (2 \times y) + 1}_{c_3}; \underbrace{y := y - 1}_{c_4}}_{c_2}}_{c_1}}_c$$

What does c do?

Let us derive a goal-oriented derivation for $\langle c, (27/x, 2/y) \rangle \longrightarrow \sigma$.

Goal-Oriented Derivation (Excerpt)



$$\begin{array}{l} \langle c, (27/x, 2/y) \rangle \longrightarrow \sigma \\ \swarrow \langle x := 0, (27/x, 2/y) \rangle \longrightarrow \sigma_1, \langle c_1, \sigma_1 \rangle \longrightarrow \sigma \\ \swarrow_{\sigma_1=(27/x, 2/y)[n/x]} \langle 0, (27/x, 2/y) \rangle \longrightarrow n, \langle c_1, (27/x, 2/y)[n/x] \rangle \longrightarrow \sigma \\ \swarrow_{n=0, \sigma_1=(0/x, 2/y)} \langle c_1, (0/x, 2/y) \rangle \longrightarrow \sigma \\ \swarrow \langle 0 \leq y, (0/x, 2/y) \rangle \longrightarrow \text{tt}, \langle c_2, (0/x, 2/y) \rangle \longrightarrow \sigma_2, \langle c_1, \sigma_2 \rangle \longrightarrow \sigma \\ \vdots \\ \swarrow_{\sigma=(9/x, -1/y)} \langle 0 \leq y, (9/x, -1/y) \rangle \longrightarrow \text{ff} \\ \swarrow^* \square \end{array}$$

Look Closely



Do you notice something odd about this rule?

$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \texttt{while } b \texttt{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \texttt{while } b \texttt{ do } c, \sigma \rangle \longrightarrow \sigma'}$$

Look Closely



Do you notice something odd about this rule?

this premise is as complex as the conclusion!

$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \texttt{while } b \texttt{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \texttt{while } b \texttt{ do } c, \sigma \rangle \longrightarrow \sigma'}$$

Look Closely



Do you notice something odd about this rule?

this premise is as complex as the conclusion!

$$\frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma'}$$

A recursive definition!

IMP Semantics (Commands)



$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \longrightarrow \sigma} \quad \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]} \quad \frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'} \\[10pt] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff} \quad \langle c_1, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \quad \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c_0, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \\[10pt] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma} \quad \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma'} \end{array}$$

A **termination property** (further expanded):

$$\blacktriangleright P(c) \triangleq \forall \sigma \in \mathbb{M}. \exists \sigma' \in \mathbb{M}. \langle c, \sigma \rangle \longrightarrow \sigma'$$

$$\forall c. P(c)?$$

Termination?



$$P(c) \triangleq \forall \sigma \in \mathbb{M}. \exists \sigma' \in \mathbb{M}. \langle c, \sigma \rangle \longrightarrow \sigma' \quad \forall c. P(c)?$$

- Consider $w \triangleq \text{while } \texttt{tt} \text{ do skip}.$

We have:

$$\langle w, \sigma \rangle \longrightarrow \sigma'$$

$$\swarrow \langle \texttt{tt}, \sigma \rangle \longrightarrow \texttt{tt}, \langle \text{skip}, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\swarrow \langle \text{skip}, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\swarrow_{\sigma''=\sigma} \langle w, \sigma \rangle \longrightarrow \sigma'$$

- We reached the same goal from which we started!

Termination?



$$P(c) \triangleq \forall \sigma \in \mathbb{M}. \exists \sigma' \in \mathbb{M}. \langle c, \sigma \rangle \longrightarrow \sigma' \quad \forall c. P(c)?$$

- Consider $w \triangleq \text{while tt do skip}$.

We have:

$$\langle w, \sigma \rangle \longrightarrow \sigma'$$

$$\swarrow \langle \text{tt}, \sigma \rangle \longrightarrow \text{tt}, \langle \text{skip}, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\swarrow \langle \text{skip}, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\swarrow_{\sigma''=\sigma} \langle w, \sigma \rangle \longrightarrow \sigma'$$

- We reached the same goal from which we started!
No options to backtrack: $\langle \text{tt}, \sigma \rangle \not\longrightarrow \text{ff}$. Cannot complete the derivation!
- We conclude $\neg P(w)$: we found a counter-example to termination.

Divergence



- ▶ $\langle c, \sigma \rangle \not\rightarrow$ means $\neg \exists \sigma'. \langle c, \sigma \rangle \rightarrow \sigma'$.
- ▶ We say that c **diverges on σ** .
- ▶ Divergence is difficult to detect and handle — it is undecidable!

Divergence: Example



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

Divergence: Example



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

► If $\sigma(x) \leq 0$:

$$\langle w, \sigma \rangle \longrightarrow \sigma' \nwarrow_{\sigma'=\sigma} \langle x > 0, \sigma \rangle \longrightarrow \text{ff} \nwarrow^* \square.$$

Hence, $\langle w, \sigma \rangle \longrightarrow \sigma$.

Divergence: Example



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

- If $\sigma(x) \leq 0$:

$$\langle w, \sigma \rangle \longrightarrow \sigma' \searrow_{\sigma'=\sigma} \langle x > 0, \sigma \rangle \longrightarrow \text{ff} \searrow^* \square.$$

Hence, $\langle w, \sigma \rangle \longrightarrow \sigma$.

- If $\sigma(x) > 0$:

$$\langle w, \sigma \rangle \longrightarrow \sigma'$$

$$\searrow \langle x > 0, \sigma \rangle \longrightarrow \text{tt}, \langle x := x + 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\searrow^* \langle x := x + 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\searrow_{\sigma''=\sigma[n/x]} \langle x := x + 1, \sigma \rangle \longrightarrow n, \langle w, \sigma[n/x] \rangle \longrightarrow \sigma'$$

$$\searrow_{n=\sigma(x)+1}^* \langle w, \sigma[\sigma(x) + 1/x] \rangle \longrightarrow \sigma'$$

Divergence: Example



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

- If $\sigma(x) \leq 0$:

$$\langle w, \sigma \rangle \longrightarrow \sigma' \not\sim_{\sigma'=\sigma} \langle x > 0, \sigma \rangle \longrightarrow \text{ff} \not\sim^* \square.$$

Hence, $\langle w, \sigma \rangle \longrightarrow \sigma$.

- If $\sigma(x) > 0$:

$$\langle w, \sigma \rangle \longrightarrow \sigma'$$

$$\not\sim \langle x > 0, \sigma \rangle \longrightarrow \text{tt}, \langle x := x + 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\not\sim^* \langle x := x + 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\not\sim_{\sigma''=\sigma[n/x]} \langle x := x + 1, \sigma \rangle \longrightarrow n, \langle w, \sigma[n/x] \rangle \longrightarrow \sigma'$$

$$\not\sim_{n=\sigma(x)+1}^* \langle w, \sigma[\sigma(x) + 1/x] \rangle \longrightarrow \sigma'$$

Not the same goal we started from! How to prove divergence?

Proving Divergence



Consider $w \triangleq \text{while } b \text{ do } c$. Proof idea: find a set of memories $S \subseteq \mathbb{M}$ such that

1. $\forall \sigma \in S. \langle b, \sigma \rangle \longrightarrow \text{tt}$
2. $\forall \sigma \in S. \forall \sigma' \in \mathbb{M}. \langle c, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S$

Then we can conclude $\forall \sigma \in S. \langle w, \sigma \rangle \nrightarrow$.

Proving Divergence



Consider $w \triangleq \text{while } b \text{ do } c$. Proof idea: find a set of memories $S \subseteq \mathbb{M}$ such that

1. $\forall \sigma \in S. \langle b, \sigma \rangle \longrightarrow \text{tt}$
2. $\forall \sigma \in S. \forall \sigma' \in \mathbb{M}. \langle c, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S$

Then we can conclude $\forall \sigma \in S. \langle w, \sigma \rangle \not\rightarrow$.

Equivalently, as a rule:

$$\frac{\sigma \in S \quad \forall \sigma \in S. \langle b, \sigma \rangle \longrightarrow \text{tt} \quad \forall \sigma \in S. \forall \sigma' \in \mathbb{M}. (\langle c, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S)}{\langle w, \sigma \rangle \not\rightarrow}$$

Note:

- If $\langle c, \sigma \rangle \not\rightarrow$ then $\langle c, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S$ trivially holds.
- Proving divergence is not always possible (nor easy)

Divergence: Example (Revisited)



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

- ▶ If $\sigma(x) \leq 0$, then we proceed to show $\langle w, \sigma \rangle \longrightarrow \sigma$ (as before).
- ▶ If $\sigma(x) > 0$, we define

$$S \triangleq \{\sigma \mid \sigma(x) > 0\}$$

Divergence: Example (Revisited)



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

- ▶ If $\sigma(x) \leq 0$, then we proceed to show $\langle w, \sigma \rangle \longrightarrow \sigma$ (as before).
- ▶ If $\sigma(x) > 0$, we define

$$S \triangleq \{\sigma \mid \sigma(x) > 0\}$$

We prove the two conditions:

1. $\forall \sigma \in S. \langle x > 0, \sigma \rangle \longrightarrow \text{tt} \checkmark$
2. $\forall \sigma \in S. \forall \sigma'. \langle x := x + 1, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S \checkmark$

Divergence: Example (Revisited)



Consider $w \triangleq \text{while } x > 0 \text{ do } x := x + 1$, and some arbitrary σ . Two possibilities:

- ▶ If $\sigma(x) \leq 0$, then we proceed to show $\langle w, \sigma \rangle \longrightarrow \sigma$ (as before).
- ▶ If $\sigma(x) > 0$, we define

$$S \triangleq \{\sigma \mid \sigma(x) > 0\}$$

We prove the two conditions:

1. $\forall \sigma \in S. \langle x > 0, \sigma \rangle \longrightarrow \text{tt} \checkmark$
2. $\forall \sigma \in S. \forall \sigma'. \langle x := x + 1, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' \in S \checkmark$

In fact:

$$\langle x := x + 1, \sigma \rangle \longrightarrow \sigma' \Rightarrow \sigma' = \sigma[\sigma(x) + 1/x]$$

$$\sigma(x) > 0 \Rightarrow \sigma[\sigma(x) + 1/x](x) = \sigma(x) + 1 > 0$$

Therefore, if $\sigma(x) > 0$ then $\langle w, \sigma \rangle \not\longrightarrow$.

IMP Semantics (Commands)



$$\begin{array}{c} \frac{}{\langle \text{skip}, \sigma \rangle \longrightarrow \sigma} \quad \frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]} \quad \frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'} \\[10pt] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff} \quad \langle c_1, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \quad \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c_0, \sigma \rangle \longrightarrow \sigma'}{\langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle \longrightarrow \sigma'} \\[10pt] \frac{\langle b, \sigma \rangle \longrightarrow \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma} \quad \frac{\langle b, \sigma \rangle \longrightarrow \text{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \text{while } b \text{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma'} \end{array}$$

We have seen **determinacy** for arithmetic expressions. For commands:

- ▶ $P(c) \triangleq \forall \sigma, \sigma_1, \sigma_2 \in \mathbb{M}. (\langle c, \sigma \rangle \longrightarrow \sigma_1 \wedge \langle c, \sigma \rangle \longrightarrow \sigma_2) \Rightarrow \sigma_1 = \sigma_2$
- ▶ $\forall c. P(c)?$

Structural Induction for Commands



Given the syntax of commands:

$$c \in Com ::= \text{skip} \mid x := a \mid c; c \mid \text{if } b \text{ then } c \text{ else } c \mid \text{while } b \text{ do } c$$

We have that structural induction is as follows:

$$\frac{\begin{array}{l} P(\text{skip}) \quad \forall x, a. P(x := a) \\ \forall c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(c_0; c_1) \\ \forall b, c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(\text{if } b \text{ then } c_0 \text{ else } c_1) \\ \forall b, c. P(c) \Rightarrow P(\text{while } b \text{ do } c) \end{array}}{\forall c \in Com. P(c)}$$

Proving Determinacy



We have two base cases:

$$P(\text{skip})$$

$$\forall x, a. P(x := a)$$

and three inductive cases:

$$\forall c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(c_0; c_1)$$

$$\forall b, c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(\text{if } b \text{ then } c_0 \text{ else } c_1)$$

$$\forall b, c. P(c) \Rightarrow P(\text{while } b \text{ do } c)$$

Proving Determinacy



We have two base cases:

$$P(\text{skip})$$
$$\forall x, a. P(x := a)$$

and three inductive cases:

$$\forall c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(c_0; c_1)$$
$$\forall b, c_0, c_1. P(c_0) \wedge P(c_1) \Rightarrow P(\text{if } b \text{ then } c_0 \text{ else } c_1)$$
$$\forall b, c. P(c) \Rightarrow P(\text{while } b \text{ do } c)$$

The case for **while** b **do** c **fails**, due to the recursive definition of its semantics.
We need a more convenient proof principle: **rule induction**.

Program Equivalence



- ▶ Symbolic manipulation of programs is possible if we can guarantee that their semantics is preserved
- ▶ Useful to study program transformations (interpreters, compilers, code optimization, refactoring, etc)
- ▶ But, when are two commands equivalent?

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Consider the commands $x := y + 1$ and $x := 1$.

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Consider the commands $x := y + 1$ and $x := 1$.

► Clearly, $x := y + 1 \sim_o x := 1$

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Clearly, $x := y + 1 \sim_o x := 1$
- ▶ Let $C[\bullet] \triangleq y := 1; [\bullet]$ be a **context**: an environment for commands.

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Clearly, $x := y + 1 \sim_o x := 1$
- ▶ Let $C[\bullet] \triangleq y := 1; [\bullet]$ be a **context**: an environment for commands.
- ▶ Given $C[\bullet]$, we can “plug in” arbitrary commands in $\bullet$. We obtain:
 $C[x := y + 1] \triangleq y := 1; x := y + 1 \quad C[x := 1] \triangleq y := 1; x := 1$

Operational Equivalence



A first attempt, using the default initial memory σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Clearly, $x := y + 1 \sim_o x := 1$
- ▶ Let $C[\bullet] \triangleq y := 1; [\bullet]$ be a **context**: an environment for commands.
- ▶ Given $C[\bullet]$, we can “plug in” arbitrary commands in $\bullet$. We obtain:
 $C[x := y + 1] \triangleq y := 1; x := y + 1$ $C[x := 1] \triangleq y := 1; x := 1$
- ▶ However, the resulting commands are **not equivalent**, as we have

$$y := 1; x := y + 1 \not\sim_o y := 1; x := 1$$

- ▶ There is a context that **does not preserve** the equivalence!

Operational Equivalence



Another attempt, using **any initial memory** :

$$c_1 \sim c_2 \triangleq \forall \sigma, \sigma'. (\langle c_1, \sigma \rangle \longrightarrow \sigma' \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma')$$

We have that this definition of $\sim$ is

- ▶ an **equivalence** (reflexive, symmetric, transitive)
- ▶ a **congruence**: it is preserved by every context!



The End